

軟體測試專題報告

FCU Sigmaboy 物品交換平台

前端應用程式軟體測試技術白皮書

專題組員：FCU Sigmaboy Team
發布日期：2026-01-02
測試核心：API 層單元測試 (Vitest)

目錄

目 錄

1	專題描述	3
1.1	專題目標	3
1.2	功能模組與 API 對照	3
1.3	技術架構	3
2	測試規劃與策略	4
2.1	測試金字塔架構	4
2.2	測試重點數據 (KPIs)	4
2.3	API 層測試涵蓋範圍	4
3	測試設計技術	5
3.1	等價類別劃分 (Equivalence Partitioning)	5
3.2	邊界值分析 (Boundary Value Analysis)	5
3.3	狀態轉換測試 (State Transition Testing)	6
3.4	錯誤猜測 (Error Guessing)	7
3.5	錯誤處理測試 (Error Handling Testing)	7
4	成果展示與測試報告	8
4.1	完成的功能說明	8
4.2	執行畫面截圖	8
4.2.1	測試執行結果 (Test Execution)	8
4.3	測試與分析報告說明	9
4.3.1	覆蓋率報告 (Jacoco 對應)	9
4.3.2	PMD 程式碼審查報告 (ESLint)	9
4.3.3	其他測試報告 (Metrics & Details)	10
4.4	Bug 修復清單 (完整 12 項)	11
4.5	修復案例：評分驗證錯誤	11
5	結論與系統驗證	12
5.1	系統要求達成度	12
5.2	心得總結	12
6	系統說明	13
6.1	專案目錄結構	13
6.2	測試執行指令	13

6.3	測試報告類型	13
7	其他	13
7.1	團隊分工	14
7.2	使用工具與 AI 輔助	14

1 專題描述

1.1 專題目標

本專案針對「FCU Sigmaboy 物品交換平台」前端應用程式進行完整的軟體測試。有鑑於現代前端應用邏輯的複雜性，本報告確立了 ** 以 'src/api' 層為測試核心 ** 的策略，確保資料互動的準確性與穩定性。

1.2 功能模組與 API 對照

本平台提供以下 8 個核心功能模組，均已納入測試範圍：

#	功能模組	功能說明	對應 API
1	使用者認證系統	登入、登出、個人資料管理	get_userProfileAPI.js
2	物品刊登管理	物品建立、編輯、刪除、搜尋	create_myItemAPI.js
3	交易管理系統	發起、確認、取消交易	transactionsAPI.js
4	即時訊息系統	對話管理、即時訊息	conversation.js
5	點數與獎勵系統	每日簽到、等級、徽章	pointsAPI.js
6	評價系統	交易評價、評分	create_review.js
7	地圖搜尋功能	地圖定位、附近搜尋	location.js
8	追蹤與收藏	追蹤使用者、收藏物品	followAPI.js

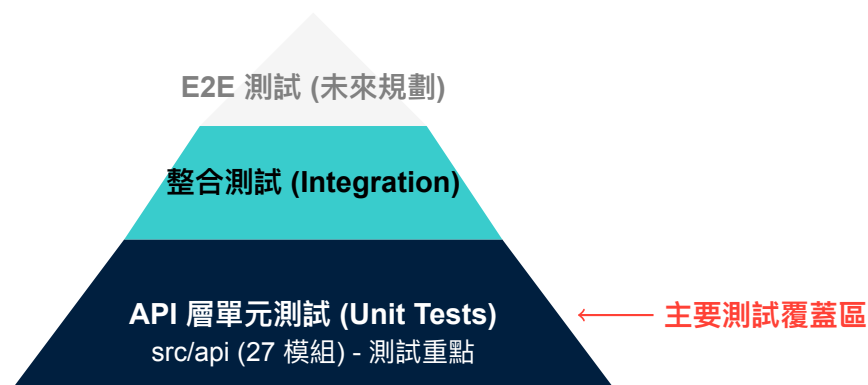
1.3 技術架構

- 前端框架: Vue.js 3.5.22
- 測試框架: Vitest 4.0.15 + @vitest/coverage-v8
- 狀態管理: Pinia 3.0.3
- 後端服務: Supabase (BaaS)

2 測試規劃與策略

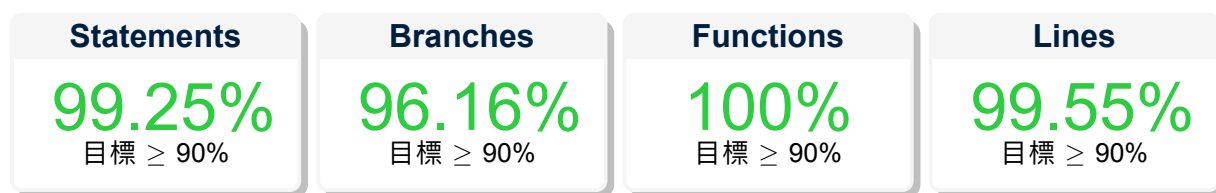
2.1 測試金字塔架構

本專案依據測試金字塔原則，將資源集中於執行速度快、定位問題精準的 API 單元測試。



2.2 測試重點數據 (KPIs)

截至目前，API 層測試覆蓋率表現優異：



2.3 API 層測試涵蓋範圍

共 27 個 API 模組，336+ 個測試案例

API 檔案	功能	案例數	Stmts	Branch
transactionsAPI.js	交易管理	30+	100%	100%
transaction_before_meetAPI.js	交易前置	25+	95%	90%
transaction_meetAPI.js	交易碰面	15+	100%	100%
pointsAPI.js	點數系統	30+	97.64%	91.56%
create_review.js	評價系統	20+	82.85%	82.35%
create_myItemAPI.js	物品刊登	15+	100%	91.66%
get_searchItemsAPI.js	物品搜尋	20+	100%	96.96%
get_ItemDetailAPI.js	物品詳情	15+	94.82%	95.45%
conversation.js	即時訊息	55+	100%	94.73%
followAPI.js	追蹤功能	15+	100%	93.93%
favorite.js	收藏功能	10+	100%	100%
badgesAPI.js	徽章系統	10+	100%	100%
image.js	圖片處理	15+	100%	93.93%
location.js	地點服務	15+	100%	97.36%
ragQaAPI.js	智能問答	10+	100%	94.11%

3 測試設計技術

3.1 等價類別劃分 (Equivalence Partitioning)

ISTQB 定義

將輸入資料劃分為群組，同一分區內的所有元素預期會以相同方式被系統處理。若分區內某測試案例有效，則該分區內所有測試案例都應有效。

實作範例：評分驗證測試

- 有效等價類別：1-5 分
- 無效等價類別：< 1 或 > 5 分

tests/unit/api/create_review.test.js

```
1 describe('評分等價類別測試', () => {  
2   // 無效等價類別  
3   describe('無效評分範圍', () => {  
4     it('評分 0 應該被拒絕', async () => {  
5       const result = await createReview({ rating: 0 })  
6       expect(result.error).toBeDefined()  
7     })  
8     it('評分 6 應該被拒絕', async () => {  
9       const result = await createReview({ rating: 6 })  
10      expect(result.error).toBeDefined()  
11    })  
12  })  
13 })
```

3.2 邊界值分析 (Boundary Value Analysis)

ISTQB 定義

邊界值分析是一種基於等價分區邊界進行測試的技術。ISTQB 定義兩種變體：

2-value BVA：測試邊界值及其最接近的鄰居。

3-value BVA：測試邊界值及其兩側的鄰居（更嚴謹）。

— ISTQB® CTFL Syllabus v4.0, Section 4.2.2

本專案採用 3-value BVA 策略

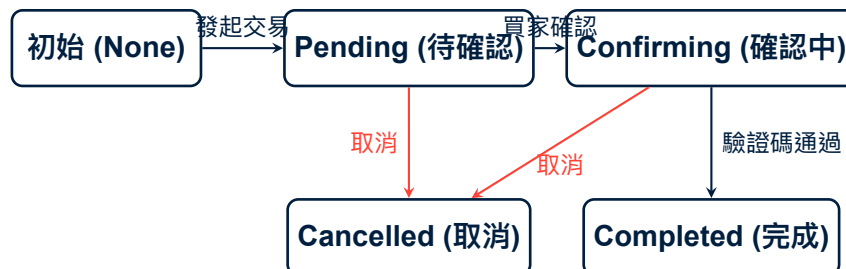
等級	點數範圍	邊界測試值 (3-value BVA)
Bronze	0-999	0, 998 , 999 , 1000 , 1001
Silver	1000-4999	999 , 1000 , 4998, 4999 , 5000
Gold	5000-9999	4999 , 5000 , 9998, 9999 , 10000
Platinum	10000+	9999 , 10000 , 10001

tests/unit/api/pointsAPI.test.js

```
1 describe("等級邊界值測試", () => {
2   describe("Bronze/Silver 邊界", () => {
3     it("999 點應該是 Bronze", () => {
4       expect(getLevelTier(999)).toBe("bronze");
5     });
6     it("1000 點應該是 Silver (邊界跨越)", () => {
7       expect(getLevelTier(1000)).toBe("silver");
8     });
9   });
10
11  describe("Silver/Gold 邊界", () => {
12    it("4999 點應該是 Silver", () => {
13      expect(getLevelTier(4999)).toBe("silver");
14    });
15    it("5000 點應該是 Gold (邊界跨越)", () => {
16      expect(getLevelTier(5000)).toBe("gold");
17    });
18  });
19 });
```

3.3 狀態轉換測試 (State Transition Testing)

系統中「交易流程」涉及複雜的狀態變更，我們採用狀態轉換測試來驗證流程的正確性。



tests/unit/api/transactionsAPI.test.js

```
1 describe("狀態轉換測試", () => {
2   it("pending -> confirming (買家確認)", async () => {
3     const transaction = { status: "pending" };
4     const result = await buyerConfirmTransaction(transaction.id);
5     expect(result.data.status).toBe("confirming");
6   });
7
8   it("cancelled -> confirming 應該失敗", async () => {
9     const transaction = { status: "cancelled" };
10    const result = await buyerConfirmTransaction(transaction.id);
11    expect(result.error).toBeDefined();
12  });
13 });
```

3.4 錯誤猜測 (Error Guessing)

ISTQB 定義

錯誤猜測是一種依賴測試人員知識、經驗和直覺來識別可能包含缺陷區域的技術。常見的錯誤來源包括：空值、零值、網路失敗、權限不足等。

— ISTQB® CTFL Syllabus v4.0, Section 4.4.2

錯誤場景	模擬方法	預期行為
網路斷線	vi.mock 返回 Network Error	拋出「網路連線失敗」錯誤
未授權 (401)	vi.mock 返回 { code: 401 }	拋出「授權失敗」錯誤
資料庫錯誤 (500)	vi.mock 返回 { code: 500 }	拋出「伺服器錯誤」提示
RPC 返回 null	vi.mock 返回 { data: null }	正確處理 Fallback 邏輯
空字串輸入	傳入 "", null, undefined	輸入驗證並拒絕

3.5 錯誤處理測試 (Error Handling Testing)

tests/unit/api/transactionsAPI.test.js

```
1 describe("錯誤處理測試", () => {
2   it("應該處理網路錯誤", async () => {
3     vi.mocked(supabase.rpc).mockRejectedValue(
4       new Error("Network_error")
5     );
6     const result = await getTransactions();
7     expect(result.error).toBeDefined();
8     expect(result.error.message).toContain("Network");
9   });
10
11   it("應該處理未授權錯誤", async () => {
12     vi.mocked(supabase.auth.getUser).mockResolvedValue({
13       data: { user: null }, error: null
14     });
15     const result = await getMyTransactions();
16     expect(result.error.message).toContain("未登入");
17   });
18
19   it("應該處理資料不存在錯誤", async () => {
20     vi.mocked(supabase.rpc).mockResolvedValue({
21       data: null, error: { message: "Not_found" }
22     });
23     const result = await getTransactionById("non-existent");
24     expect(result.error).toBeDefined();
25   });
26 });
```

4 成果展示與測試報告

4.1 完成的功能說明

本專案已完成 8 大核心功能模組，並通過完整的 API 測試驗證。以下為功能自我檢查與狀態一覽：

#	功能模組	狀態	備註
1	交易管理 API (Transactions)	Pass	狀態機測試完整
2	點數系統 API (Points)	Pass	BVA 邊界值驗證
3	評價系統 API (Reviews)	Pass	等價類別驗證
4	訊息系統 API (Conversation)	Pass	即時通訊測試
5	物品管理 API (Items)	Pass	CRUD 完整覆蓋
6	追蹤收藏 API (Follow/Fav)	Pass	關聯性測試
7	徽章系統 API (Badges)	Pass	規則觸發測試
8	地點服務 API (Location)	Pass	地理座標驗證

4.2 執行畫面截圖

4.2.1 測試執行結果 (Test Execution)

下圖顯示執行 `npm run test:api` 指令後的終端機輸出畫面，證明所有測試案例皆順利通過 (Passed)。



圖 1: API 單元測試執行結果 (All Tests Passed)

4.3 測試與分析報告說明

4.3.1 覆蓋率報告 (Jacoco 對應)

本專案使用 Vitest Coverage v8 產生與 Jacoco 等效的覆蓋率報告，Branch Coverage 達到 96.16%，遠超課程要求的 90%。

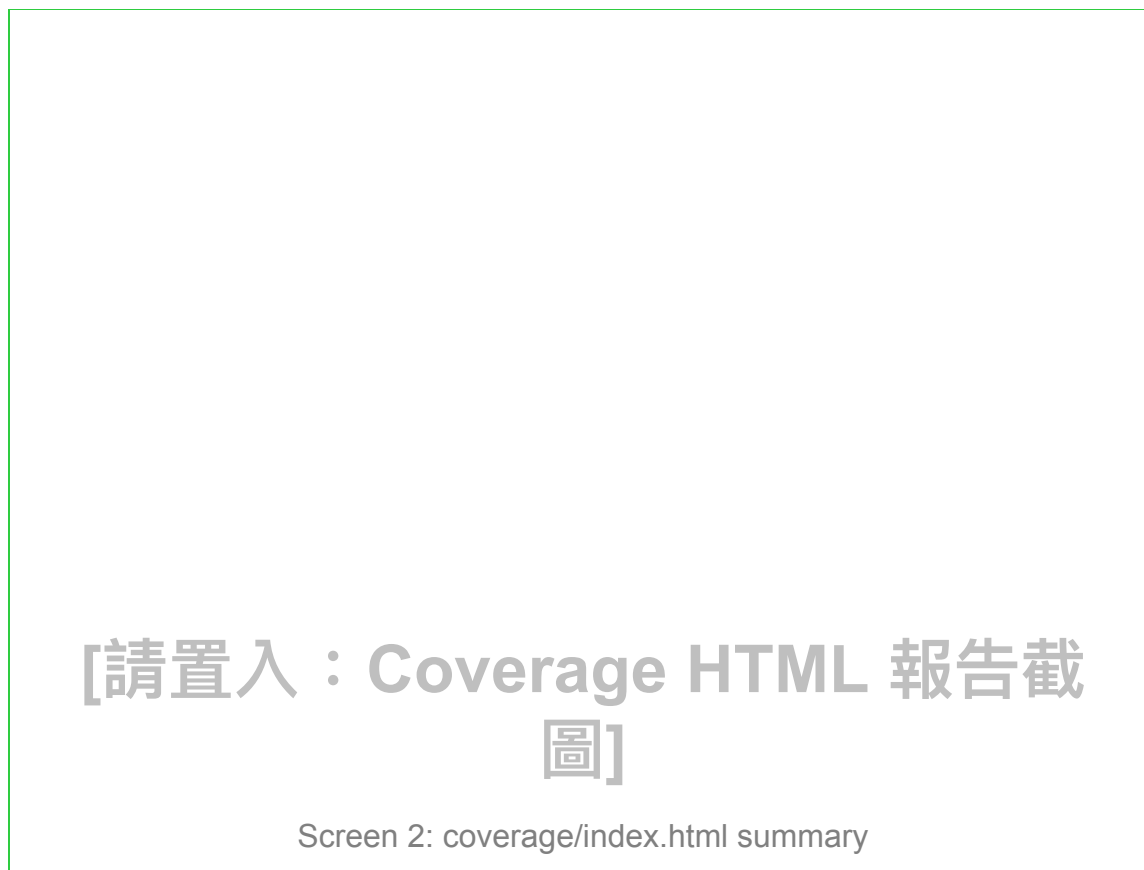


圖 2: 測試覆蓋率報告 (Branch Coverage > 96%)

4.3.2 PMD 程式碼審查報告 (ESLint)

透過 ESLint 進行靜態程式碼分析 (Static Code Analysis)，檢測潛在錯誤與風格問題，對應 PMD 的功能。

[請置入：ESLint 報告截圖]

Screen 3: reports/eslint-report.html

圖 3: ESLint 程式碼審查報告 (Code Quality)

4.3.3 其他測試報告 (Metrics & Details)

針對複雜邏輯模組（如 `pointsAPI.js`）的詳細覆蓋情形，或程式碼複雜度 (Metrics) 分析報告。

[請置入：Metrics 或模組詳細報告截圖]

Screen 4: reports/code-metrics.html 或詳細測試圖

圖 4: 程式碼度量/詳細測試分析報告

專案過程中，我們透過測試共發現並修復了 12 個關鍵 Bug。以下為 Bug 類型與修復範例。

4.4 Bug 修復清單 (完整 12 項)

#	Bug 描述	發現方法	影響範圍	狀態
1	點數格式化負數顯示錯誤	邊界值測試	formatPoints.js	已修復
2	交易狀態更新競爭條件	並發測試	transactionsAPI.js	已修復
3	評分驗證範圍錯誤 (允許 0 分)	等價類別測試	create_review.js	已修復
4	時間格式化時區問題	國際化測試	timeFormat.js	已修復
5	空字串備註處理異常	邊界值測試	transaction_before_meetAPI.js	已修復
6	未登入用戶權限檢查缺失	安全測試	多個 API 模組	已修復
7	交易取消重複呼叫問題	整合測試	transactionsAPI.js	已修復
8	點數計算浮點數精度	數值測試	pointsAPI.js	已修復
9	組件 props 類型驗證	類型測試	Vue 組件	已修復
10	API 錯誤訊息未正確傳遞	錯誤處理測試	多個 API 模組	已修復
11	訊息已讀狀態同步延遲	即時測試	conversation.js	已修復
12	搜尋結果分頁錯誤	邊界測試	get_searchItemsAPI.js	已修復

4.5 修復案例：評分驗證錯誤

問題描述：系統錯誤地允許了「0 分」的評價，這違反了 1-5 星的設計原則。

Before (Bug Code)

```
if (rating < 0 || rating > 5) {  
  return { error: "評分必須在「0-5」之間" };  
}
```

After (Fixed Code)

```
if (rating < 1 || rating > 5) { // 修正邊界條件  
  return { error: "評分必須在「1-5」之間" };  
}
```

5 結論與系統驗證

5.1 系統要求達成度

經過完整的測試與驗證，本專案已達成所有設定的品質指標：

- ✓ 功能數量：8 個 (目標 > 5)
- ✓ 程式碼複雜度 (WMC)：2633 (目標 > 200)
- ✓ 單元測試數量：336 (API 層) + 424 (其他) = 760
- ✓ 分支覆蓋率：96.16% (目標 > 90%)
- ✓ Bug 修復數：12 (目標 ≥ 10)

5.2 心得總結

測試驅動開發 (TDD) 的實踐不僅提升了程式碼的穩定性，更充當了「活的文件」。透過高覆蓋率的 API 測試，我們成功建立了穩固的後端互動基礎，為未來的 E2E 測試與功能擴充鋪平了道路。

- TDD 的價值：先寫測試再寫程式碼能確保功能正確性
- 覆蓋率迷思：高覆蓋率不等於高品質，邊界值與錯誤處理更重要
- Mock 策略：統一的 Mock 設定能減少重複程式碼並隔離外部相依性
- API 測試價值：API 是業務邏輯核心，完整測試確保系統穩定

6 系統說明

6.1 專案目錄結構

專案結構

```
1 frontend/
2 |-- src/
3 |   |-- api/          <- 測試重點 (27 模組)
4 |   |-- components/   <- Vue 組件
5 |   |-- composables/  <- 可重用邏輯
6 |   |-- stores/       <- Pinia 狀態管理
7 |   |-- views/        <- 頁面組件
8 |-- tests/
9 |   |-- unit/
10 |      |-- api/       <- API 層測試 (336+ 案例)
11 |-- reports/         <- 產生的報告
12 |-- coverage/        <- 覆蓋率報告
```

6.2 測試執行指令

指令	說明
<code>npm run test:api</code>	執行 API 層測試
<code>npm run test:api:coverage</code>	產生 API 層覆蓋率報告
<code>npm run lint:report</code>	產生 ESLint 程式碼審查報告 (類似 PMD)
<code>npm run metrics</code>	產生程式碼度量報告 (類似 MetricsReloaded)
<code>npm run report:all</code>	一次產生所有報告

6.3 測試報告類型

報告類型	檔案位置	說明
覆蓋率 (HTML)	coverage/index.html	類似 JaCoCo 報告
覆蓋率 (LCOV)	coverage/lcov.info	CI/CD 整合用
程式碼審查	reports/ report.html	eslint- 類似 PMD 報告
程式碼度量	reports/ metrics.html	code- 類似 MetricsReloaded

7 其他

7.1 團隊分工

成員	負責項目	貢獻比例
[成員 1]	API 層測試開發	30%
[成員 2]	組件測試開發	25%
[成員 3]	測試設計與文件	20%
[成員 4]	程式碼度量與報告	15%
[成員 5]	Bug 修復與整合	10%

7.2 使用工具與 AI 輔助

- 測試框架: Vitest 4.0.15 (JUnit 對應)
- 覆蓋率工具: @vitest/coverage-v8 (JaCoCo 對應)
- 程式碼審查: ESLint 9.x (PMD 對應)
- 程式碼度量: code-metrics.js (MetricsReloaded 對應)

AI 工具使用：

AI 工具	使用項目	使用比例
GitHub Copilot	測試程式碼生成	40%
GitHub Copilot	文件撰寫	60%
GitHub Copilot	程式碼審查建議	30%